Optimization of Privacy Budget Allocation In Differential Privacy-based Public Transit Trajectory Data Publishing for Smart Mobility Applications

Chenxi Chen, Xianbiao Hu*, Yang Li, Qing Tang

***Abstract*—Trajectory datasets have been widely used in transportation research, but the risk of privacy breach comes with data sharing. Privacy budget allocation is a key step of the differential privacy (DP)-based privacy-preserving data publishing (PPDP) algorithm development, as it directly impacts the data utility of the released dataset. Most prior research used simple logic to allocate privacy budgets, such as evenly distributing them among different tree levels, without theoretical support to reach optimality. This manuscript presents the development of an optimal privacy budget allocation algorithm for transit smart card data, with the goal of publishing non-interactive sanitized trajectory data under a differential privacy definition. To this end, the smart card trajectory data are first stored in a prefix tree structure, and a query probability model is developed to quantitatively measure the probability of a trajectory location pair being queried. Next, the privacy budget is optimized for each prefix tree node to minimize the query error, while satisfying the differential privacy definition. The Lagrangian relaxation method is adopted to derive the optimal privacy budget values, and several propositions on the solution property are proposed and proved. Real-life metro smart card data from Shenzhen, China that include a total of 2.8 million individual travelers and over 220 million records are used in the case study section. The developed algorithm is demonstrated to output a sanitized dataset with the highest utilities when compared with three benchmark algorithms. Sensitivity analysis shows that the resulting data utility remains stable when the privacy budget changes. The runtime of the proposed algorithm is less than 160 seconds in all experiments, exhibiting good computational efficiency.**

***Index Terms*—Privacy protection, transit trajectory data, privacy budget, Lagrangian relaxation**

I. INTRODUCTION

TRAJECTORY datasets, such as those collected by GPS devices or smart cards, are widely used in transportation research [1, 2]. Data sharing is essential to enable research, and it is important to share trajectory datasets after removing personal explicit identifier (ID), such as the name of a traveler or the license plate number of a vehicle.

The risk of privacy breach comes with data sharing, even after the removal of personal information. For example, according to a research study [3], as long as a person's 5-digit ZIP code, date of birth and gender is known, the likelihood to uniquely identify them in the US is as high as 87%. This implied that with some simple background knowledge, it was technically feasible to identify a person from the database. In fact, the health records (which included diagnoses and prescriptions) of William Weld, then Governor of Massachusetts, were uniquely identified from the "anonymized" healthcare database and sent to his office in a theatrical flourish. Ironically, the governor himself was the champion of the data sharing and assured the public that the government had protected patient privacy by deleting identifiers [4]. In transportation, a similar privacy breach risk exists when making trajectory datasets public. For example, an early study based on smart card datasets [5] found that 30.7% of users can be uniquely identified, as long as a traveler's two card swipe records were known. In the following trajectory dataset from Shenzhen, China (Fig. 1), each column stores the travel history of one traveler, with pseudo identifier as the unique ID. One would think that after the removal of personal information (e.g., name), it should be safe to release such a dataset publicly. However, this is not the case. For example, if Alice chooses to post her location information on social media (such as in a Facebook or Twitter post), saying she was at a subway station named "Buxin" at 7:25 a.m., and then checked into "Longcheng Square" station at 8:14 a.m., a hacker can easily identify Alice in the publicly shared dataset, as 20015461 was the only traveler who had been to these two stations at these times. The hacker can then query Alice's complete travel history in the past. For example, where did Alice go over the weekend, and did Alice go home after work last Wednesday.

Data utility and privacy protection thus become two conflicting objectives of the privacy-preserving data publishing (PPDP) problem, when releasing a trajectory dataset. On one hand, if the original dataset is released without any privacy protection method, the utility is the highest, but the dataset is prone to privacy attacks. On the other hand, if we emphasize solely the privacy protection, the risk of attack becomes lowest, but the dataset may not represent the original statistical pattern and thus loses its values. Differential privacy (DP), first proposed in [6], provides a quantified measure of privacy loss and an upper bound and allows curators to choose the explicit tradeoff between privacy and accuracy. A key parameter of DP algorithm, ε, refers to the privacy budget and can quantitatively describe the degree of such tradeoff.

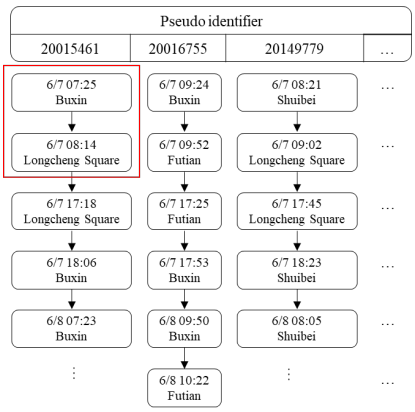


Fig. 1. Sample transit trajectory data

While research on ε-differential privacy has been active over the last decade, how to optimally allocate the privacy budget ε among the entire dataset has not been given much attention. Many existing research efforts have allocated a privacy budget equally on different layers, after representing the trajectory data in a tree structure. Representative examples include the SeqPT model [7, 8] and SafePath model [9]. Such a simple budget allocation mechanism, although easy to implement, ignores the data characteristics and may not allocate the privacy budget in a cost-effective way. In one of the research team's recent studies [5], it was argued that due to the tree structure's characteristics, when the depth increased, the number of nodes reduced, so the same privacy budget would cause more significant impact to the data quality. As such, an incremental privacy budget allocation model was proposed, in which the privacy budget of each node was calculated by its layer in the tree and increased from the top down. The detail of the model will be explained in section II. This was because that, if the privacy budget was fixed the same, when the count value increased, the risks of privacy breach would be reduced. While such an incremental budget allocation method is intuitive and has been demonstrated to improve data utility, the model lacks theoretical support to reach optimality and thus may not lead to the optimal privacy budget allocation solution.

To bridge such a gap, this manuscript aims to formulate the budget allocation problem in the privacy-preserving data publishing process as an optimization model, so that the developed optimal budget allocation model would lead to not only a theoretically sound but also a verifiable improved solution. To do so, we adopt the same prefix tree structure to store the trajectory data as discussed in [5] and develop a query probability model to quantitatively measure the probability of a trajectory location pair being queried. The rationale is that the sensitivity of outcomes caused by adding noises depends on the trajectory location pair frequencies. If a location pair is queried more frequently, the impact of adding the same amount of noise would become more significant. As such, in the next steps we then allocate privacy budget for each prefix tree level to minimize the query error, weighted by the query probability of all nodes at that tree level. Finally, the problem is formulated with the Lagrangian relaxation method. Several propositions on the solution property are proposed and proved, and the solution algorithm is presented.

II. LITERATURE REVIEW

The collection of digital information has created tremendous opportunities for knowledge- and information-based decision making. This is especially the case in the transportation domain, as open dataset (e.g., trajectory data) enables many transportation research opportunities. As pointed out in [10], data in its original form, however, typically contains sensitive information about individuals, and the publication of such data will violate individual privacy. To deal with the privacy challenges, privacy-preserving data publishing provides methods and tools for publishing useful information while preserving data privacy.

Syntactic privacy models are widely utilized in trajectory data privacy preserving, such as k-anonymity, l-diversity, t-closeness, or β-likeness. The basic idea of syntactic privacy models is to generalize the database entries until some syntactic condition is met, so that the ability of an adversary to link a quasi-identifier tuple to sensitive values is restricted [11]. Additionally, suppression-based model is another popular method to achieve *k*-anonymity. However, compared with the flourishing research in the computer science domain, PPDP research in the field of transportation engineering has been very limited, and the data needs for transportation modeling and privacy protection should be deliberately balanced for different applications [12]. This is indeed important and adds significance to the PPDP research in transportation domain. Hoh et al. [13] proposed a system based on virtual trip lines (VTL) and an associated cloaking technique to maximize the accuracy of real-time traffic estimation while protecting privacy. In another study, part of the location traces from mobile sensors was filtered out in the VTL zone-based system by several techniques, including random sampling, and individual probability-based and entropy-based approaches [12]. A *k*-anonymity-based privacy control algorithm was proposed to allow mobility operators to share data [14]. Tour data were used as an example, where an operator's data were used in conjunction with performance measure accuracy controls to synthesize a set of alternative tours with diffused probabilities for sampling during a query, and tests were conducted on a set of routing records from travel data in Long Island, NY. In another study, Ghasemzadeh et al. [15] proposed an *LK*-anonymity model to achieve anonymity in a trajectory database while preserving the information to support effective passenger flow analysis. A license plate recognition (LPR) dataset was studied in [16] based on *k*-anonymity. It was found that with as few as five spatiotemporal data records, 90% of individuals can be uniquely identified, and to quantify the tradeoff between privacy and data utility, suppression and generalization solutions were proposed.

To the best of our knowledge, the existing PPDP studies in the transportation domain are not only limited, but are also mostly based on syntactic privacy models, as reviewed above. However, syntactic privacy models assume that an attacker's background knowledge is known in advance, whereas in reality, it is literately impossible to know what background knowledge a hacker possesses before the attack is initiated. As a result, syntactic privacy models are not effective against privacy attacks, especially when hackers have some degree of background knowledge on the attacked dataset. To address these issues, Dwork [6] first proposed differential privacy, or DP, as a strict definition of the uninformativeness privacy principle, and it does not require assumptions of what background knowledge a hacker possesses.

Applications of differential privacy on trajectory datasets can be found in many sources in the literature. SeqPT [8] was the first tree-based PPDP algorithm, in which the sequential locations were organized as a prefix tree from root node to leaf node. Each node recorded the sub-sequence frequent pattern, in which the Laplacian noises were added. Though it was claimed that SeqPT can be extended to trajectory data, Li et al. [5] showed that this work was not suitable for large trajectory datasets. Compared with sequential data, trajectory data also contained the time dimension besides the spatial dimension, which led to an exponentially increased dimension and thus made direct application of DP on a trajectory dataset

challenging. Built upon SeqPT, Al-Hussaeni et al. [9] developed a model called SafePath, which generated a new noisy prefix tree that was ε-differntially private, by introducing a variable height and degree tree with location and timestamps categorized. A real-life transit data experiment [5] suggested that SafePath had significantly higher efficiency and scalability with respect to large and sparse data scenarios when compared with earlier research.

As pointed out in our earlier work [5], most DP algorithms (such as SeqPT and SafePath), after storing the trajectory dataset as a prefix tree, allocated privacy budget evenly on each layer. In other words, at each level lv, from top down, the budget was evenly distributed $\theta_{lv} = \theta_{lv+1}$. However, due to the differences in the statistical characteristics at each layer, evenly allocating the budget among different tree layers was problematic. As such, a model to incrementally assign privacy budget was proposed in [5], which we will refer to as Li's algorithm below. In this previous work, privacy budget ε_{lv} and threshold θ_{lv} of each tree layer lv were calculated by two function:$\varepsilon_{lv} = [\lg(lv+\sigma) / \sum_{lv=1}^{h} \lg(lv+\sigma)] \times \varepsilon, \sigma > 0$, and $\theta_i = k \times i^{-1} + b$, $k > 0$, $b > 0$ where σ, k, b were three adjustable parameters, ε was the total privacy budget, and h was the tree height. Based on such a model, privacy budget function ε_{lv} increased by level, from the top down, i.e., there existed $\varepsilon_{lv} < \varepsilon_{lv+1}$, as under the same privacy budget, the higher the count value was, the smaller the impact would be due to the added noise. On the other hand, because the average count value on nodes decreased by tree level, the threshold function θ_{lv} also decreased by level, $\theta_{lv} > \theta_{lv+1}$, $1 \le lv < h$. Such model, although intuitive to understand, lacked theoretical support to reach optimality and thus may not lead to the optimal privacy budget allocation solution.

III. PRELIMINARIES

A. ε-differential privacy

Definition 1 (Differential Privacy). A randomized mechanism M gives ε-differential privacy, if for any two datasets $\mathcal{D}_1$ and $\mathcal{D}_2$ that are different by only one record, and for any new sanitized dataset $\widehat{\mathcal{D}} \in Range(M)$, the following Eq. (1) is always satisfied:

$$\Pr[M(\mathcal{D}_1) = \widehat{\mathcal{D}}] \le \exp(\varepsilon) \times \Pr[M(\mathcal{D}_2) = \widehat{\mathcal{D}}] \quad (1)$$

M is a randomized algorithm which takes a dataset as input and generates the protected dataset. The parameter ε stands for the privacy budget and is a positive number. ε typically ranges between 0 and 1, with 0 indicating highest privacy protection level and with most noises added to the original dataset. The idea behind differential privacy is that if the effect of making an arbitrary single substitution in the database is small enough, the query result cannot be used to infer much about any single individual, and therefore provides privacy. This is achieved by adding Laplace noise (i.e., noise coming from the Laplace distribution) that satisfies the definition.

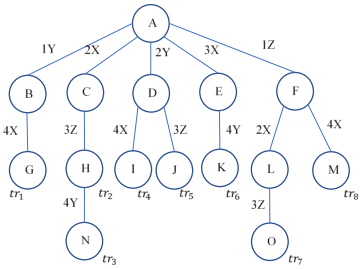
B. Representing trajectory data as a prefix tree

We use tr_i to denote the ith trajectory data, with t_i as the timestamp and loc_i as the location. $\mathcal{T}$ is the time domain and $\mathcal{L}$ is the location domain, so we have $t_i \in \mathcal{T}$ and $loc_i \in \mathcal{L}$. It's intuitive to see that in one trip, t_i should be strictly increasing in the sequence. We use $|tr|$ to denote the trajectory length, i.e., the number of timestamp and location pairs in a single trip. In Fig. 2(a) we show a total of eight sample trajectory data. Most of them have a length of 2 (e.g., $|tr_1| = 2$), meaning the travelers checked into (or out of) the subway station two times, while the third and seventh trajectory have a length of 3 (i.e., $|tr_3| = 3$ and $|tr_7| = 3$).

Next, we show how to represent trajectory data as a prefix tree, which is a commonly seen tree structure to organize structural data. Since our study target is trajectory data, which is a type of spatial-temporal sequence data, the tree structure becomes a good form to store them. In the designed prefix tree, we use the arc to store a check-in record from the smart card dataset, so that any two consecutive arcs form a trajectory location pair in the original trajectory data. For example, in Fig. 2(b), arc AB represents 1Y, and arcs AB and BG form a trajectory location pair 1Y → 4X, i.e., trajectory tr_1 is now represented by arcs AB and BG, or nodes $A \to B \to G$. Node A becomes the only root node in Fig. 2(b), as it is the topmost node in the tree and does not have any parent nodes. Leaf nodes include G, N, I, J, K, O, and M, as they don't have any child nodes. The nodes that have the same parent nodes (such as I and J, and L and M) become sibling nodes.

ID	Trajectory
tr_1	1Y→ 4X
tr_2	2X→ 3Z
tr_3	2X→ 3Z→ 4Y
tr_4	2Y→ 4X
tr_5	2Y→ 3Z
tr_6	3X→ 4Y
tr_7	1Z→ 2X→ 3Z
tr_8	1Z→ 4X

(a)



(b)

Fig. 2. Representing trajectory data as prefix tree. (a) sample trajectory data; (b) prefix tree representation.

After storing all trajectory data in the prefix tree structure, we use the node count to store the number of incoming trajectory data. For example, for node B in Fig. 2(b), the number of incoming trajectory data 1Y is 1 (the only trajectory data that matches with this pattern is tr_1), so we have count(B)=1; whereas for node C we have count(C) = 2, as both tr_2 and tr_3 include location pair 2X → 3Z. The maximum tree height is 4, and the longest trees are $A \to C \to H \to N$, or $A \to F \to L \to O$. The shortest tree length is 3. It should be noted that Fig. 2 merely reflects the example of a simple and small dataset, for illustration purposes, whereas in reality the dataset could be much larger and the tree will be much more complex.

C. Relative error

While DP offers a privacy guarantee, our interest in this manuscript is to maximize the data utility of the published dataset, under the differential privacy definition. As such, we define relative error as the following: to quantitively measure the impact of the developed algorithm on the trajectory dataset. In Section V, we will use this definition to measure and compare the effectiveness of the algorithm proposed in this manuscript with the three benchmark models from literature.

Definition 2 (Relative Error). Relative error on a sanitized dataset $\widehat{\mathcal{D}}$ is defined by equation $relative_{error} = |q(\widehat{\mathcal{D}}) -$

$q(\mathcal{D})\big|/\max\{q(\mathcal{D}), s\}$ [5, 9]. q is a count query function and returns the number of sub-trajectories in a trajectory dataset. $q(D)$ is the count query result of the original dataset $\mathcal{D}$ and $q(\widehat{\mathcal{D}})$ represents the count query result of the sanitized dataset $\widehat{\mathcal{D}}$. This equation represents how different the sanitized dataset $\widehat{\mathcal{D}}$ is from the real trajectory dataset D. s in the equation represents the sanity bound, the goal of which is to avoid that the relative error is dominated by small value of $q(\mathcal{D})$ [17]. The sanity bound value is usually set to be 0.1% of the dataset in many research on differential privacy [5, 7, 8, 18] and the sensitivity analysis regarding the relative error change with the different sanity bound value will be introduced in Section V.

IV. METHODOLOGY

In the section below, we present a new algorithm for privacy-preserving trajectory data publishing based on differential privacy. The input data are the original trajectory dataset $\mathcal{D}$ and a pre-defined privacy budget ε, and the output data are the sanitized trajectories $\widehat{\mathcal{D}}$ that follow the differential privacy definition. We first use the same method as Li et al. [5] to convert raw trajectory data $\mathcal{D}$ into the prefix tree structure, and allocate the total privacy budget ε to each tree node. Then, we add Laplace distributed noise according to the generated budgets, and finally, we traverse and output sanitized trajectories $\widehat{\mathcal{D}}$ from the sanitized tree.

In the subsections below, we describe how to allocate the total privacy budget ε to each tree node at different levels so as to maximize the data utility in $\widehat{\mathcal{D}}$. An important step to consider is how to quantify the impact of allocating certain privacy budget to a tree node. Two main aspects are considered. The first is how frequently is this node (or the associated incoming and outgoing arc pair) likely to be queried, and the second is the expectation of introduced error by adding Laplace noises. To this end, in subsection A we propose a query probabilistic model to estimate the probability of a node being queried, in subsection B we formulate the studied problem as an optimization model with constraints, and we discuss some important propositions of this model in subsection C. In subsection D, we describe the key steps of the algorithm with pseudocode, and finally in subsection E, theoretical analysis is performed to discuss its improvements and privacy guarantee.

A. Query probabilistic model

This subsection describes a query probability model to compute the probability of a location being queried. To start off, a few assumptions in trajectory query generation are listed as follows:

1. Queries are generated randomly so that all possible trajectory patterns might be selected with a uniform probability. This assumption follows differential privacy definition that we don't need to assume background knowledge that a hacker may have, and instead, all locations (or subway stations in our dataset) have equal chances of being queried in a single query event.
2. Each location in the trajectory data is sampled from the entire location set with replacement. However, following typical mobility behavior patterns, two adjacent locations in a trajectory set cannot be the same. In other words, we assume a typical traveler will move from point A to point B, instead of moving from point A to point A, which forms a loop and doesn't happen frequently in reality.
3. Each trajectory query is generated independently. In other words, there is no correlation between multiple queries, and we can treat these queries as independent events.

Based on these assumptions, we first calculate the total number of possible query patterns that a hack may generate. We use $|\mathcal{L}|$ to denote the total number of distinct locations in the trajectory dataset $\mathcal{D}$, and use l_{max} and l_{min} represent the length of the longest and shortest trajectory, respectively.

If we set the length of query to be l, with $l_{min} \le l \le l_{max}$ number of different trajectory patterns with a length of l is derived in Eq. (2). This is because due to assumption 1, all trajectory patterns will be randomly generated and enumerated with replacement, so the first location in the query has $|\mathcal{L}|$ possibilities. Further, following assumption 2, two adjacent locations in a trajectory set cannot be the same, so each location in the remaining query will have $|\mathcal{L}| - 1$ possibilities.

$$\begin{aligned} N_l &= |\mathcal{L}| * (|\mathcal{L}| - 1) * (|\mathcal{L}| - 1) * \dots * (|\mathcal{L}| - 1) \\ &= |\mathcal{L}| * (|\mathcal{L}| - 1)^{l-1} \end{aligned} \quad (2)$$

For example, say we have three locations, X, Y, and Z from a trajectory dataset with $l_{min} = 1$ and $l_{max} = 3$. If the query length l is set to be 1, the set of trajectory data that might be queried by a hacker will include [X, Y, Z], with $N_1 = 3$ (which equals to $|3| * (|3| - 1)^{1-1}$). If l is set to be 2, the set of trajectory data that might be queried includes [XY, XZ, YX, YZ, ZX, ZY] with $N_2 = 6$ (which equals to $|3| * (|3| - 1)^{2-1}$). [XX, YY, ZZ] are not good query patterns, as they conflict with assumption 2.

So, for trajectory length l, the total number of possible trajectory query patterns can be calculated by Eq. (3).

$$N = \sum_{l=l_{min}}^{l_{max}} N_l = \frac{|\mathcal{L}| * (|\mathcal{L}|-1)^{l_{min}-1}\left[(|\mathcal{L}|-1)^{l_{max}-l_{min}+1}-1\right]}{|\mathcal{L}|-2} \quad (3)$$

Next, we compute the probability of each trajectory being queried in the prefix tree. It should be noted that, as illustrated in the [X, Y, Z] three-location example above, one single query may involve multiple locations, or in other words, multiple trajectories in the prefix tree might be queried in one trajectory query pattern. For example, in Fig. 2(b), if a hacker queried trajectory location pair 1Y → 4X, both arcs AB and BG (i.e., nodes A, B, G) will be queried. As such, the probability of each trajectory being queried in the prefix tree does not automatically equal to $1/N$. Next, we will show how to derive the probability of each node in the prefix tree being queried.

We use a bottom-up approach, and first examine the trajectories associated with the leaf nodes, i.e., the trajectories with longest length $l = l_{max}$his is the simplest scenario. As there is only one possible trajectory from the leaf to a root node, the probability of these trajectories being queried becomes $1/N$, derived via Eq. (4).

$$p_{leaf} = \frac{1}{N} = \frac{|\mathcal{L}|-2}{|\mathcal{L}| * (|\mathcal{L}|-1)^{l_{min}-1}\left[(|\mathcal{L}|-1)^{l_{max}-l_{min}+1}-1\right]} \quad (4)$$

Let's use the prefix tree in Fig. 2(b) as an example. As A is the root node, and G is a leaf node, there is only one possible query pattern that involves both A and G, which is A-B-G (or 1Y→4X in the original trajectory dataset). Similarly, only one possible query pattern exists from the root node to the other leaf nodes, such as A→N, A→I, and so on.

Then, we move up one level along the prefix tree, i.e., tree level $l = l_{max} - 1$. If the studied node has k child nodes (i.e., if the node i has k outgoing arcs), the probability of node i being queried is equal to the probability of itself being queried, plus that of any of its children nodes being queried. For example, in Fig. 2(b), node B is involved in both queries of A-B-G and A-B, and node D is involved in both queries of A-D-I and A-D-J. With this, the probability of a non-leaf node i being queried is shown in Eq. (5), in which p_{itself} is the probability of itself being queried, and p_{child} is that of its child queried.

$$p_i = p_{itself} + k * p_{child} = 1/N + k/N \tag{5}$$

Finally, if we continue to move up levels along the prefix tree, Eq. (5) can be iteratively applied to calculate the probability of any node in the tree being queried. In general, for each node in the prefix tree, the probability of it being queried equals the probability of itself being queried, plus that of any of its children being queried. So iteratively, Eq. (6) can be derived for any node i in the prefix tree:

$$p_i = p_{itself} + \sum_{j \epsilon child(i)} p_j = 1/N + \sum_{j \epsilon child(i)} p_j \tag{6}$$

B. Optimal privacy budget allocation algorithm

The Laplace mechanism is arguably the most important algorithm in differential privacy. It is a symmetric version of the exponential distribution and adds noise from a symmetric continuous distribution to the true answer. In our context, the Laplace mechanism is used to generate noise, which will then be added to the node count in the prefix tree.

In a general form, if we assume X follows the Laplace distribution, with $X \sim Laplace(\mu, b)$, the probability density function of X is:

$$f(x|\mu, b) = \frac{1}{2b} e^{\frac{-|x-\mu|}{b}} \tag{7}$$

The expectation value of Laplace distribution $E(X) = \mu$, and the variance $D(X) = 2b^2$.

In the context of differential privacy, if we assign a privacy budget of ε_i to a tree node i, the expectation of the error brought by adding Laplace noise $Err = 2/{\varepsilon_i}^2$.For a node i, if Laplace noise with privacy budget ε_i is added, since the probability of this node being queried is p_i, the expectation of total query error that is introduced by adding Laplace noises $E\big(Err(i)\big) = p_i * Err = 2p_i/{\varepsilon_i}^2$

As such, the objective function of our problem can be formulated as Eq. (8), with the goal of minimizing the query error after adding noises, or in other words, to maximize the data utility.

$$minimize\ f(\varepsilon_i) = \sum_i E\big(Err(i)\big)/2 = \sum_i p_i/{\varepsilon_i}^2 \tag{8}$$

For the constraints of this optimization model, we know that starting from the root to any leaf node j, the summation of privacy budget along the path should be equal to ε. This is because the path from root node to leaf node represents a complete trajectory, and per the sequential composition property of differential property, the resulting query is $\sum_{i=1}^{n} \varepsilon_i$-differentially private, which leads to Eq. (9).

$$\sum_{i \epsilon path(root,j)} \varepsilon_i = \varepsilon \qquad \forall j \epsilon leaf \tag{9}$$

Again, if we use the prefix tree in Fig. 2(b) as an example, since there are 7 leaf nodes, we will have a total of 7 constraints as listed below, each representing a path from the root node to a leaf node.

$$\begin{gathered} \varepsilon_G + \varepsilon_B + \varepsilon_A = \varepsilon \\ \varepsilon_N + \varepsilon_H + \varepsilon_C + \varepsilon_A = \varepsilon \\ \dots \\ \varepsilon_M + \varepsilon_F + \varepsilon_A = \varepsilon \end{gathered} \tag{10}$$

With this, the optimal budget allocation model is now formulated, with its constraints, as summarized in Eq. (11).

$$\begin{gathered} minimize\ \ f(\varepsilon_i) = \sum_i E\big(Err(i)\big) = \sum_i \frac{p_i}{{\varepsilon_i}^2} \\ s.t. \qquad \sum_{i \epsilon path(root,j)} \varepsilon_i = \varepsilon \qquad \forall j \epsilon leaf \\ p_i = \frac{1}{N} + \sum_{j \epsilon child(i)} p_j \\ p_{leaf} = \frac{|\mathcal{L}|-2}{|\mathcal{L}|*(|\mathcal{L}|-1)^{l_{min}-1}\left[(|\mathcal{L}|-1)^{l_{max}-l_{min}+1}-1\right]} \end{gathered} \tag{11}$$

If we convert the above problem with the Lagrangian relaxation method, the original optimization model will lead to Eq. (12), which is now a non-constrained optimization model.

$$\begin{gathered} minimize\ L(\varepsilon_i) = \sum_i \frac{p_i}{{\varepsilon_i}^2} + \\ \sum_{j \epsilon leaf} \lambda_j \big(\sum_{i \epsilon path(root,j)} \varepsilon_i - \varepsilon\big) \end{gathered} \tag{12}$$

Next, we derive some important properties of this privacy budget allocation model, based on Eq.(12).

We start by examining a leaf node i. To obtain the optimal privacy budget ε_i, we can compute the first derivative of Eq. (12) and make it equal to 0. Eq. (13) is thus obtained for each of the leaf nodes.

$$\frac{dL(\varepsilon_i)}{d\varepsilon_i} = -2 * \frac{p_i}{{\varepsilon_i}^3} + \lambda_i = 0 \tag{13}$$

For example, for the prefix tree in Fig. 2(b), we have a total of 7 equations derived as below, each for a leaf node.

$$\begin{gathered} -2 * \frac{p_G}{{\varepsilon_G}^3} + \lambda_G = 0 \\ -2 * \frac{p_N}{{\varepsilon_N}^3} + \lambda_N = 0 \\ \dots \\ -2 * \frac{p_M}{{\varepsilon_M}^3} + \lambda_M = 0 \end{gathered} \tag{14}$$

Now, we move on to the non-leaf nodes. Again, we calculate the first derivative of Eq. (12) and make it equal to 0, and will obtain Eq. (15) for each of the non-leaf nodes. $child\ leaf(i)$ refers to a list of leaf nodes that stem from the examined non-leaf node i.

$$\frac{dL(\varepsilon_i)}{d\varepsilon_i} = -2 * \frac{p_i}{{\varepsilon_i}^3} + \sum_{j \epsilon child\ leaf(i)} \lambda_j = 0 \tag{15}$$

For example, for the prefix tree in Fig. 2(b), we have a total of 7 equations derived as below, each for a non-leaf node.

$$\begin{gathered} -2 * \frac{p_B}{{\varepsilon_B}^3} + \lambda_G = 0 \\ -2 * \frac{p_D}{{\varepsilon_D}^3} + (\lambda_I + \lambda_J) = 0 \\ \dots \\ -2 * \frac{p_A}{{\varepsilon_A}^3} + \big(\lambda_G + \lambda_N + \lambda_I + \lambda_J + \lambda_K + \lambda_O + \lambda_M\big) = 0 \end{gathered} \tag{16}$$

C. Propositions of the solution property

Before we present the solution algorithm, we examine Eqs.(13) and (15) and discuss and prove a few useful propositions, which will make the problem solving much easier.

Proposition 1: if two nodes in the prefix tree are sibling nodes and they both are non-leaf nodes, the summation of privacy budget for any of their child tree is equal.

Mathematically, for non-leaf sibling nodes i and j, if we use $child\ leaf(i)$ and $child\ leaf(j)$ to represent the leaf nodes that are stemming from node i and j, respectively, we will have:

$$\sum_{a \epsilon path(i,child\ leaf(i))} \varepsilon_a = \sum_{a \epsilon path(j,child\ leaf(j))} \varepsilon_a \tag{17}$$

Proof: we use p to denote the parent of i and j (since they are siblings, they share the same parent node), and use tr_i and tr_j to denote the trajectories that are associated with i and j, respectively.

Following differential privacy's sequential composition property, for any complete trajectory data, starting from the root node to its leaf node, the summation of privacy budgets that are allocated to the nodes along the path should equal to ε. In other words, we will have Eqs. (18) and (19),

$$\sum_{a\in path(root,p)} \varepsilon_a + \sum_{a\in path(i,child\ leaf(i))} \varepsilon_a = \varepsilon \quad (18)$$

$$\sum_{a\in path(root,p)} \varepsilon_a + \sum_{a\in path(j,child\ leaf(j))} \varepsilon_a = \varepsilon \quad (19)$$

By comparing Eqs. (18) and (19), the first item and the item on the right side of the equation are the same, as such we have $\sum_{a\in path(i,child\ leaf(i))} \varepsilon_a = \sum_{a\in path(j,child\ leaf(i))} \varepsilon_a$ and thus the proof is now complete.

Proposition 2: if two nodes are sibling nodes and they both are leaf nodes, their privacy budgets are equal.

$$\varepsilon_i = \varepsilon_j \text{ if } i \text{ and } j \text{ are sibling leaf nodes} \quad (20)$$

The proof of this property is straightforward. Eqs. (18) and (19) continue to hold, except that since i and j are sibling leaf nodes, i is the only node in $path(i, child\ leaf(i))$, and j is the only node in $path(j, child\ leaf(j))$. As such, Eq. (18) becomes $\sum_{a\in path(root,p)} \varepsilon_a + \varepsilon_i = \varepsilon$, and Eq. (19) becomes $\sum_{a\in\in path(root,p)} \varepsilon_a + \varepsilon_j = \varepsilon$. By comparing these two equations, we can determine $\varepsilon_i = \varepsilon_j$ and the proof is now complete.

Proposition 3: For any non-leaf node i, if it covers a total of k children leaf nodes $childleaf_{i1}, childleaf_{i2} \dots childleaf_{ik}$, the following equation holds:

$$\frac{p_i}{{\varepsilon_i}^3} = \sum_{a\epsilon[1,k]} \frac{p_{childleaf_{ia}}}{{\varepsilon_{childleaf_{ia}}}^3} \quad (21)$$

From the Lagrangian relaxation equation (13), we can derive Eq. (22), which should hold for any leaf node.

$$\lambda_i = 2{p_i}/{{\varepsilon_i}^3} \quad (22)$$

Then, use Eq. (22) to replace the λ_i in Eq. (15), and with some simple conversion, we will have Eq. (23) for any non-leaf node:

$$\frac{p_i}{{\varepsilon_i}^3} = \sum_{a\epsilon[1,k]} \frac{1}{2}\lambda_a = \sum_{a\epsilon[1,k]} \frac{p_{childleaf_{ia}}}{{\varepsilon_{childleaf_{ia}}}^3} \quad (23)$$

The proof is now complete.

With this property, we can calculate ε_i for any non-leaf node i as expressed in Eq. (24) below.

$$\varepsilon_i = \left(p_i / \sum_{a\epsilon[1,k]} \frac{p_{childleaf_{ia}}}{{\varepsilon_{childleaf_{ia}}}^3}\right)^{\frac{1}{3}} \quad (24)$$

These three propositions make the problem solving much easier. For example, for the prefix tree in Fig. 2(b), following the first proposition, we will have Eq. (25) – (a) and (b). Following the second proposition, we will have Eq. (25) – (c). Following the third proposition, we will have a few more constraints, including Eq. (25)–(d)~(f). Together, they form the equation set that allows us to find a solution to the proposed model.

$$\begin{aligned}
&\varepsilon_L + \varepsilon_O = \varepsilon_M && \text{(a)}\\
&\varepsilon_B + \varepsilon_G = \varepsilon_C + \varepsilon_H + \varepsilon_N = \varepsilon_D + \varepsilon_I = \varepsilon_E + \varepsilon_K = \varepsilon_E + \varepsilon_L + \varepsilon_O && \text{(b)}\\
&\varepsilon_I = \varepsilon_J && \text{(c)}\\
&\frac{p_B}{{\varepsilon_B}^3} = \frac{p_G}{{\varepsilon_G}^3} && \text{(d)}\\
&\dots && \dots
\end{aligned} \quad (25)$$

$$\frac{p_F}{{\varepsilon_F}^3} = \frac{p_M}{{\varepsilon_M}^3} \quad \text{(e)}$$

$$\frac{p_A}{{\varepsilon_A}^3} = \frac{p_G}{{\varepsilon_G}^3} + \frac{p_N}{{\varepsilon_N}^3} + \frac{p_I}{{\varepsilon_I}^3} + \frac{p_J}{{\varepsilon_J}^3} + \frac{p_K}{{\varepsilon_K}^3} + \frac{p_O}{{\varepsilon_O}^3} + \frac{p_M}{{\varepsilon_M}^3} \quad \text{(f)}$$

D. Solution algorithms

In this subsection, we present the solution algorithms to the proposed PPDP problem. Algorithm 1 is the main algorithm and includes four main components: building a tree, allocating budgets, adding noise, and outputting data.

In Algorithm 1 Line 1, we firstly scan the original transit trajectory dataset $\mathcal{D}$ once, to build prefix tree $\mathcal{PT}$. h is the height of the tree. Next, we calculate the optimized budgets in Algorithm 1, Line 2, and allocate them to each node with Procedure 1. After that, in Lines 3-11 we add the noise to $\mathcal{PT}$, from the top of the tree to the bottom, until a new differential private prefix tree is built. Finally, Line 12 generates the new sanitized trajectory dataset $\widehat{\mathcal{D}}$ as the final output data.

Algorithm 1. Main Function

Input: Raw transit trajectory $\mathcal{D}$
Input: Prefix tree height h
Input: Total privacy budget that is available ε
Input: Threshold parameters k, b
Output: Sanitized trajectory dataset $\widehat{\mathcal{D}}$
1. Scan $\mathcal{D}$ once to build $\mathcal{PT}$ and store $\mathcal{D}$ with the tree structure;
2. $\mathcal{PT} \leftarrow$ BudgetAllocation ($\mathcal{PT}$, ε)
3. Start from root node with tree level $lv = 1$;
4. **While** $lv \le h$ do:
5. Compute the privacy threshold of level lv, $\theta_{lv} = k \times (lv)^{-1} + b$;
6. **For** each node i in level lv **of** $\mathcal{PT}$ **do**:
7. $i.count$ += Lap($1/\varepsilon_i$)
8. BuildChildTree ($i, \varepsilon_i, c_i, \theta_{lv}$);
9. **end for**
10. $lv++$;
11. **end while**
12. $\widehat{\mathcal{D}} \leftarrow$ Output ($\mathcal{PT}$); return $\widehat{\mathcal{D}}$.

The most important part of the algorithm is to optimize the privacy budget allocation, which is implemented in Procedure 1. The input is a prefix tree and the total budget ε. The output is a sanitized tree with allocated budget ε_i for each node. Two recursive functions are included in the procedure. The first one allocates budgets for the leaf nodes (Procedure 1, Lines 1-4) as well as for the nodes whose every children node has been allocated with an optimized budget with Eq. (24) (Procedure 1, Lines 5-16). $i.children_{count}$ is the count of all children nodes of node i. The second function *Normalize* $(v_i, coef)$ ensures that the sum of allocated budgets in each path is always the same, i.e. Eq. (17) is always satisfied (Procedure 1, Lines 17-24). It is designed to make sure that for any trajectory, starting from its leaf node to the root node, the summation of privacy budget added to the nodes along the path should always be equal to ε.

Procedure 1. BudgetAllocation

Input: Root node of prefix Tree $\mathcal{PT}$, total budget ε
Output: Sanitized Prefix Tree $\mathcal{PT}'$ with optimized budget ε_i
1. **Function** Allocate (i)
2. **If** i is **a** leaf node **then**
3. $\varepsilon_i = \varepsilon$
4. **break**
5. **else**
6. **For** each children node j (of node i), **do**
7. Allocate (j)
8. **end for**

9. $sum = 0$
10. **For** each leaf node $leaf_i$ (that stem from node i), **do**
11. $sum \mathrel{+}= \varepsilon/{\varepsilon_{leaf_i}}^3$
12. **end for**
13. $\varepsilon_i = i.(children_{count}/sum^{\frac{1}{3}})$
14. Normalize $(i, \varepsilon_i/\varepsilon + 1)$
15. **end if**
16. **end function**
17. **Function** Normalize $(i, coef)$
18. $\varepsilon_i \neq coef$
19. **if** i is not a leaf node, **then**:
20. **for** each children node j (of node i), **do**
21. Normalize $(j, coef)$
22. **end for**
23. **end if**
24. **end function**
25. Start budget allocation from the root node of $\mathcal{PT}$: Allocate $(root)$

Another step in the main function is to select out-edges e_{out} of each node i, so as to build its child tree. Procedure 2 is designed for this purpose. First, in Line 3 we use the calculated privacy budget ε_i and add noise to the tree nodes. After that, in Lines 4–7 we retain a node if its noisy count is no less than threshold θ_{lv}, otherwise, the node will be removed from the tree. After all nodes are scanned, in Line 12, we check if the total noise on all existing child nodes is less than c_i, which stands for the parent node's noise count value. If yes, we will randomly select more timestamp and location pairs. It should be noted that these timestamp and location pairs need to be reasonable, meaning, traveling from one node to the other in the timestamp-location pairs should be feasible, i.e., it should be feasible for a traveler to use transit to get to point B from point A, within the given amount of time, between this timestamp-location pair. The reachable location matrix $\mathcal{RL}$ in Line 12-27 is designed for this purpose.

In Line 17, we set the count value on the selected nodes as Laplace noise plus zero, and check if the count value is higher than, or equal to θ_{lv}. If yes, in Line 17-19, we add the node to the child node set. For the newly selected nodes that has a beginning count value of 0, we call them "empty nodes". When an empty node is chosen, we add the noise count to the output sum, as specified in Lines 20-21. We then decide when to stop growing the tree through the accumulation of counts on the child nodes. In Lines 23-25, the loop will end if the value of summation is no less than the count value $i.count$.

Procedure 2. BuildChildTree

Input: Parent node i, location l_i, and time t_i; threshold θ_{lv}, privacy budget ε_i, noisy count c_i; reachable location matrix $\mathcal{RL}$
Output: Noisy children nodes set $\mathcal{N}$

1: $sum = 0$;
2: **For** each child node j (of node i), **do**
3. $c_j \mathrel{+}= \text{Lap}\left(\frac{1}{\varepsilon_i}\right)$;
4. **If** $c_j \geq \theta_{lv}$ **then**
5. $\mathcal{N} \leftarrow j$;
6. $sum \mathrel{+}= c_j$;
7. **end if**
8. If $sum \geq c_j$ **then**
9. **break**;
10. **end if**
11. **end for**
12. **While** $sum < c_i$ **do**
13. Randomly select a time $t_j \in \mathcal{T}$ with $t_j > t_i$
14. Find reachable location set $\mathcal{L}_r$ from l_i in time $(t_j - t_i)$
15. **For** each location $l_j \in \mathcal{L}_r$ do
16. $count = \text{Lap}(1/\varepsilon_i) + 0$
17. **If** $count \geq \theta_{lv}$ **then**
18. Add $e_{out} = t_j\ l_j$ as a new outgoing edge;
19. Add node j as a new child node;
20. $c_j = count$;
21. $sum \mathrel{+}= count$;
22. **end if**
23. **If** $sum \geq c_i$ **then**
24. **break**;
25. **end if**
26. **end for**
27. **end while**
28. **return** $\mathcal{N}$;

E. Privacy guarantee

Algorithm 1 consists of four steps: BuildRawTree, BudgetAllocation, BuildChildTree, and Output. The first step, BuildRawTree, generates the prefix tree from the original spatiotemporal trajectory dataset, and the second step, BudgetAllocation, only calculates the budgets of all nodes. These two steps don't consume any privacy budget.

In the third step, BuildChildTree, a noise prefix tree is built iteratively, one level at a time, based on the output data of the first step BuildRawTree. Here, the parallel composition theorem applies, as the nodes on the same tree level contain a disjoint set of trajectories, whereas the sequential composition property applies to the nodes at different levels. Since the noisy prefix tree has a height of h, BuildChildTree consumes the privacy budget in an amount of $\sum_{i=1}^{h} \varepsilon_i$. Any tree branch, i.e., from the root to any leaf node, represents a complete trajectory, and according to Eq. (9), the resulting query is $\sum_{i=1}^{n} \varepsilon_i$ - differentially private. In other words, the summation of privacy budgets along any trajectory is equal to the total budget.

The last step outputs the sanitized prefix tree to trajectories and it merely changes the data structure, so no privacy budget is consumed in this step. As such, in conclusion, ε-differential privacy is guaranteed in Algorithm 1 when given a specific parameter ε as the privacy budget.

V. NUMERICAL EXPERIMENT

This section analyzes the utility and efficiency of the proposed algorithm, with the same dataset as Li et al. [5]. The real-life dataset is collected from the Shenzhen Metro smart card records and records the payment history of travelers who boarded and/or alighted from transit vehicles. The dataset covers 2.8 million smart card users with over 220 million records. A sample dataset can be found in Fig. 1

As with the study by Li et al. [5], we extract four datasets with different data size $|\mathcal{D}|$, time domain $|\mathcal{T}|$, location domain $|\mathcal{L}|$, maximum trajectory length $\max|tr|$, and average trajectory length $avg|tr|$, for algorithm evaluation. The sizes of the four datasets are listed in Table I

TABLE I EXPERIMENTAL DATASET STATISTICS

Dataset	$\lvert\mathcal{D}\rvert$	$\lvert\mathcal{T}\rvert$	$\lvert\mathcal{L}\rvert$	$\max\lvert tr\rvert$	$\text{avg}\lvert tr\rvert$
1	393,552	16	121	6	1.84
2	772,606	48	121	16	3.56
3	824,957	64	121	18	3.55
4	845,727	80	121	20	3.73

Three models from the literature, namely SeqPT [8], SafePath [9] and Li's algorithm [5], were chosen as the

benchmark models to compare with the results of our proposed algorithm. Each benchmark algorithm was implemented by the research team, and the results were verified to duplicate, or at least very closely mirror, the results published in their own respective papers. In the next few subsections, we perform comparisons with the benchmark models as well as evaluate the data utility sensitivity and the efficiency and scalability of the proposed algorithm.

A. Comparison with other models

This section compares our algorithm's data utility and runtime efficiency with the three benchmark models. We use the average relative error of the count queries to measure the data utility, in which we use 0.1% of the dataset size for the sanity bound s. Fig. 3 presents the average relative error difference under the different prefix tree heights of four datasets. The X-axis represents different prefix tree heights and is set to be between 2 and 5. The Y-axis represents the average relative error.

It can be seen that in Fig. 3, compared with the other three curves, the red curve, which represents the results from our proposed model, is always the lowest, meaning the proposed model is able to achieve the highest data utility. In the first subfigure, SeqPT (in black color) has the highest error, i.e., the worst data utility, where the other three curves are very close to each other. For the remaining three datasets, the red curve is clearly shown to be below the other curves. The black curve disappears in these three subfigures, as SeqPT fails to generate a sanitized tree when the data size increases. The relative error of the proposed algorithm ranges between 0.008 and 0.076, i.e., 0.8%~7.6%, which indicates a satisfactory data utility.

Next, we examine and compare their computational efficiencies. Again, we generate four subgraphs to show the runtime results under four different datasets, and in each subfigure, use four lines to represent the results from the four algorithms. In general, the computational efficiency of the developed algorithm (in red color) is better than SeqPT (in black color) and is close to SafePath (in blue color) but is slower than Li's algorithm (in purple color). SeqPT has the highest run time in dataset 1 and fails to run for the three datasets with larger data sizes. SafePath has similar runtime with the developed algorithm. In general, when the tree height is small, the proposed algorithm runs faster than SafePath; however, when the tree sizes get large, SafePath becomes faster.

In all four subgraphs, the developed algorithm is able to finish in 160 seconds, or within 3 minutes. Considering that most non-interactive PPDP problems are offline tasks, such runtime should be acceptable for most applications. Another interesting observation is that as tree height increases, the algorithm run time increases linearly, which is the sign of a good property and indicates the algorithm's practicality when dealing with larger datasets. The developed algorithm requires a bit more time than Li's algorithm, which is understandable, as it needs to perform an extra step to calculate the optimal privacy budget for each node.

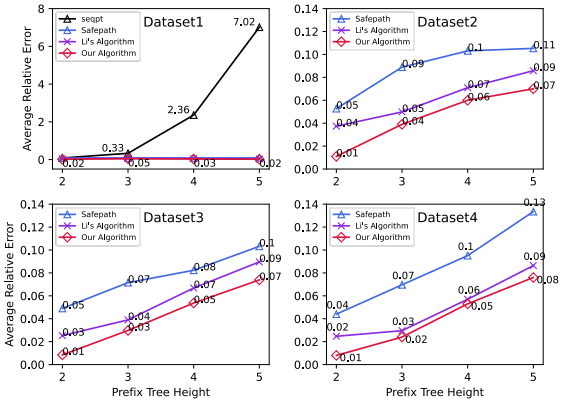


Fig. 3. Average relative error under different tree heights.

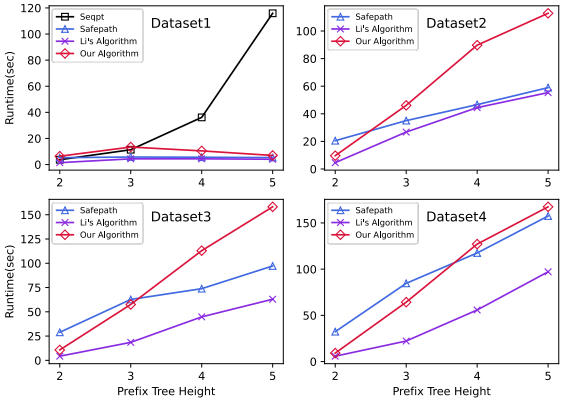


Fig. 4. Runtime comparison under different heights.

B. Data utility sensitivity analysis

This section is designed to perform sensitivity analysis on data utility, on different datasets and with different tree heights and privacy budgets. As with the previous evaluation method as described in Li et al. [5], 40,000 random count queries of length $|q| = 2$ are generated to examine our algorithm to evaluate its utility. The results are shown in Fig. 5, in which the X-axis represents different prefix tree heights and is set to be between 4 and 14. The Y-axis represents the average relative error. The four lines in each subgraph represent $\varepsilon = 0.5, 0.75, 1.0, 1.25$, respectively.

It can be observed that when tree height increases, a clear decreasing trend is observed in dataset 1, whereas dataset 2 does not fluctuate much. Data utility in dataset 3 first increases then decreases, and dataset 4 also has a peak in between. Such observations indicate that the data utility does not strictly increase or decrease with the increase of tree height. Such a pattern is consistent with prior research, such as [5], in which query error generally decreases but sometimes does not show a clear pattern. This observation indicates the importance of algorithm calibration to identify the optimal tree height based on the characteristics of the dataset. Additionally, analysis is also performed to understand the relationship between privacy budget and average relative error. When the total budget increases from 0.5 to 0.75, 1.0, and 1.25, the relative error gradually increases from 0.0398 to 0.0404, 0.0406, and 0.0417, respectively. Such observation is consistent with our expectation that a higher privacy budget will lead to lower data utility.

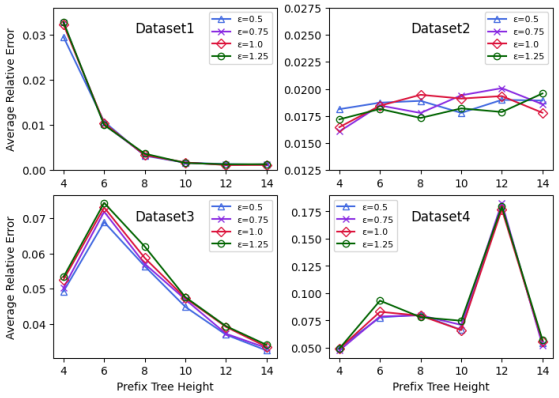


Fig. 5. Average error comparison under different tree heights.

C. Scalability analysis

In this subsection we focus on the scalability analysis of the developed algorithm. In Fig. 6(a) we break down the system runtime to four components, including reading, allocation, sanitization, and writing. In the first component, the time to read the raw dataset and build prefix tree is counted as the reading time. The time spent on building a noisy prefix tree is counted as sanitization time. In the third component, the writing time include those to traverse the noisy prefix tree and output sanitized trajectory data. The parameter settings are as following: $k = 1.5$, $b = 1$, and $\varepsilon = 1$.

In Fig. 6(a), the X-axis represents different datasets and the Y-axis represents runtime. The algorithm is found to be efficient, as even on the largest dataset (dataset4), the total run time is still less than 100 seconds. It can also be seen that among the different datasets, the runtimes on budget allocation (purple) and writing (green) do not change significantly. As a dataset gets larger, reading time (blue) increases only slightly, and data sanitization (red) is the task that costs the most time and increases significantly. Among the total time spent, about 75% of the time is spent on sanitization, 15% of the time is spent on converting raw trajectory data into prefix tree, whereas the remaining two tasks only consume about 10% of the total time.

We also examine the runtime efficiency when parameters k and b change. These are the two parameters that affect the count threshold to filter out empty nodes (algorithm 1, line 5). It can be found from Fig. 6(b) and (c) that, in general, as the values of k and b increase, the run time reduces. The runtime reduction is more significant for datasets 2, 3, and 4 than for dataset 1. In other words, for the larger dataset, tuning the k and b parameters can result in more significant runtime reductions, whereas the benefits on the smaller-scale dataset are not noticeable. It should also be noted that as parameter b continues to drop, the runtime may increase again. Finally, we look at the impact of privacy budget ε to the runtime and find that it does not change much when ε increases. Subfigures (b), (c), and (d) also demonstrate the importance of model calibration, so that the most appropriate parameters can be used to enhance the computational efficiency of the developed algorithm.

Finally, sensitivity analysis of sanity bound value is performed and the result is shown in Fig. 7. Fig. 7 (a) presents the cumulative distribution of query result /data size of Dataset1. Referring to [1], we use the 60-percentile value to determine our sanity bound, which corresponds to 0.1%. Fig. 7 (b) shows as the sanity bound decreases from 0.01% to 1%, the average relative error drops from 0.1 to 0.01. Although different sanity bounds may result in different relative errors, a same acceptable bound is adopted for different algorithms to ensure that results are comparable. The value of 0.1% is also widely used in many research on differential privacy [2-5].

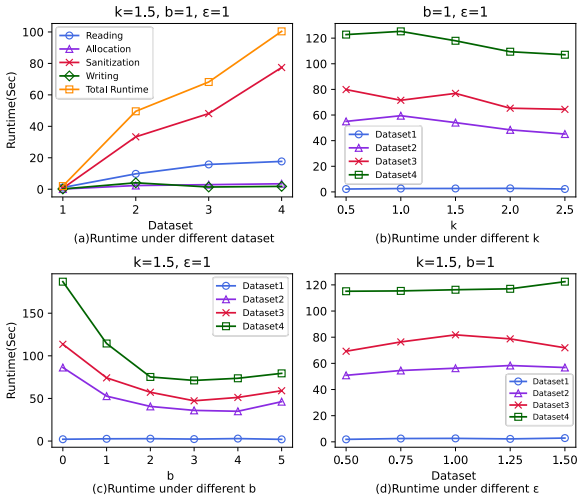


Fig. 6. Scalability analysis.

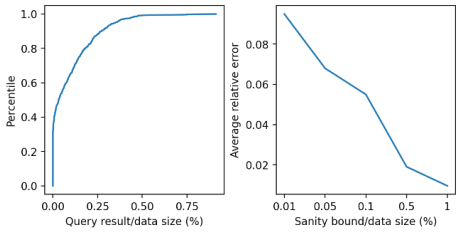


Fig. 7. sensitivity analysis of sanity bound value (a) cumulative distribution; (b) sensitivity analysis results

VI. CONCLUSIONS

In this manuscript, we focus on the optimization of budget allocation on trajectory data publishing with ε- differential privacy. We argue that most prior research used simple logic to allocate the privacy budgets, such as evenly distributing them among different tree levels, without theoretical support to reach optimality. As such, an optimal privacy budget allocation algorithm is developed for transit smart card data, with the goal of publishing non-interactive sanitized trajectory data under a differential privacy definition. An optimal privacy budget allocation model is formulated, the Lagrangian relaxation method is adopted to derive the optimal privacy budget values, and several propositions on the solution property are proposed and proved. Through theoretical analysis and comparisons with previous work based on the same real-life trajectory datasets, the sanitized trajectory dataset is shown to have the highest utility, and in the meantime satisfactory runtime efficiency.

Our proposed approach can be applied to protect data privacy in reality. After we build and add noise to the prefix tree, it can then be transferred to trajectories again and released to public. As we proved in section IV.E, the privacy was guaranteed as ε-differentially private. Moreover, data utility is improved

compared with three benchmark models, i.e., a more accurate dataset will be obtained and can be applied for other studies.

REFERENCES

1. Hu, X., *Developing the Analysis Methodology and Platform for Behaviorally Induced System Optimal Traffic Management*, In *Civil Engineering and Engineering Mechanics*. 2013, University of Arizona.

2. Song, Y., X. Hu, J. Lu, and X. Zhou, *Analytical approximation and calibration of roundabout capacity: A merging state transition-based modeling approach.* Transportation Research Part B: Methodological, 2022. **163**.

3. Sweeney, L., *Simple demographics often identify people uniquely.* Health (San Francisco), 2000. **671**: p. 1-34.

4. Anderson, N. *"Anonymized" data really isn't—and here's why not*. 2009; Available from: https://arstechnica.com/tech-policy/2009/09/your-secrets-live-online-in-databases-of-ruin/.

5. Li, Y., D. Yang, and X. Hu, *A differential privacy-based privacy-preserving data publishing algorithm for transit smart card data.* Transportation Research Part C: Emerging Technologies, 2020. **115**: p. 102634.

6. Dwork, C. *Differential privacy*. In *International Colloquium on Automata, Languages, and Programming*. 2006. Springer.

7. Chen, R., G. Acs, and C. Castelluccia. *Differentially private sequential data publication via variable-length n-grams*. In *Proceedings of the 2012 ACM conference on Computer and communications security*. 2012. ACM.

8. Chen, R., B.C. Fung, B.C. Desai, and N.M. Sossou. *Differentially private transit data publication: a case study on the montreal transportation system*. In *Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining*. 2012.

9. Al-Hussaeni, K., B.C. Fung, F. Iqbal, G.G. Dagher, and E.G. Park, *SafePath: Differentially-private publishing of passenger trajectories in transportation systems.* Computer Networks, 2018. **143**: p. 126-139.

10. Fung, B.C., K. Wang, R. Chen, and P.S. Yu, *Privacy-preserving data publishing: A survey of recent developments.* ACM Computing Surveys (Csur), 2010. **42**(4): p. 1-53.

11. Clifton, C. and T. Tassa. *On syntactic anonymity and differential privacy*. In *2013 IEEE 29th International Conference on Data Engineering Workshops (ICDEW)*. 2013. IEEE.

12. Sun, Z., B. Zan, X. Ban, and M. Gruteser, *Privacy protection method for fine-grained urban traffic modeling using mobile sensors.* Transportation Research Part B: Methodological, 2013. **56**: p. 50-69.

13. Hoh, B., T. Iwuchukwu, and Q. Jacobson, *Enhancing Privacy and Accuracy in Probe Vehicle-Based Traffic Monitoring via Virtual Trip Lines %J IEEE Transactions on Mobile Computing.* IEEE Transactions on Mobile Computing, 2012. **11**(5): p. 849-864.

14. He, B.Y. and J.Y.J. Chow, *Optimal privacy control for transport network data sharing.* Transportation Research Part C: Emerging Technologies, 2019.

15. Ghasemzadeh, M., B.C. Fung, R. Chen, and A. Awasthi, *Anonymizing trajectory data for passenger flow analysis.* Transportation research part C: emerging technologies, 2014. **39**: p. 63-79.

16. Gao, J., L. Sun, and M. Cai, *Quantifying privacy vulnerability of individual mobility traces: A case study of license plate recognition data.* Transportation Research Part C: Emerging Technologies, 2019. **104**: p. 78-94.

17. Garofalakis, M. and A. Kumar, *Wavelet synopses for general error metrics.* ACM Transactions on Database Systems (TODS), 2005. **30**(4): p. 888-928.

18. Xiao, Y., L. Xiao, X. Lu, H. Zhang, S. Yu, and H.V. Poor, *Deep-reinforcement-learning-based user profile perturbation for privacy-aware recommendation.* IEEE Internet of Things Journal, 2020. **8**(6): p. 4560-4568.



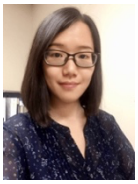
Chenxi Chen is a Ph. D. student in Transportation Engineering at Pennsylvania State University. He received Bachelor degree in Civil Engineering from Zhejiang University, Hangzhou, China and Master's degree in Transportation Engineering from Missouri University of Science and Technology, Rolla, MO. His research is focus on traffic data analytics and autonomous vehicle behavior simulation.



Xianbiao Hu is currently an Assistant Professor with the Pennsylvania State University, in its Civil and Environmental Engineering Department. He received the Ph. D. degree from the University of Arizona, Tucson, AZ in 2013. His current research is focus on smart mobility systems, connected and automated vehicles, electric vehicles, mobility behavior management, transportation big data analytics, and traffic flow and system modeling.



Yang Li is currently an Associate Professor with Guangdong University of Technology, Guangzhou, China. She was a visiting scholar at Missouri University of Science and Technology, Rolla, MO between 2018 and 2019.



Qing Tang is a Ph. D. student in Transportation Engineering at Pennsylvania State University. She received Bachelor degree in Information and Computing Sciences from Hubei University of Education, China, and Master's degree in Insurance in Risk Management from Central University of Finance and Economics, China. Her research focuses on the Autonomous Vehicles, and traffic flow and system modeling.